## *ComicCraft - AI Comic Story Creator using Gemini Models*

## *Project Description:*

The application takes in user information—such as story prompt, main character name, setting, tone, and art style—and generates a panel-by-panel storyline and corresponding illustrations. Users can view the comic preview directly on the web interface and also download the full comic in PDF format, structured with both images and narration.

With a clean, interactive HTML interface and efficient image generation powered by Hugging Face's Diffusers library, ComicCraft ensures that each story feels cohesive and visually engaging. It offers both API endpoints and an intuitive frontend for ease of use, making comic creation accessible even to non-artists and casual users.

## *Scenarios:*

*Scenario 1:* A user enters a prompt describing their comic story, such as "A brave fox exploring an enchanted forest." The system uses this input to generate a complete comic strip, including multiple panels with accompanying images and text. The user also specifies preferences like the main character's name, the setting (e.g., forest), tone (e.g., dramatic), and art style (e.g., anime). The system combines Gemini 1.5 Pro for story generation and Stable Diffusion for image creation to produce a visually immersive, personalized comic strip. This personalized approach enhances user satisfaction by generating a cohesive comic story that fits the user's creative vision.

*Scenario 2:* A user wants a light-hearted, cartoonish feel for their comic. On the input form, they select "funny" as the tone and "comic book" as the art style (instead of the default or their previous choices). When they submit, the system regenerates the entire pipeline—outline, story, and images—this time with prompts and model instructions tailored to produce a more humorous narrative and a classic comic-book look. This lets

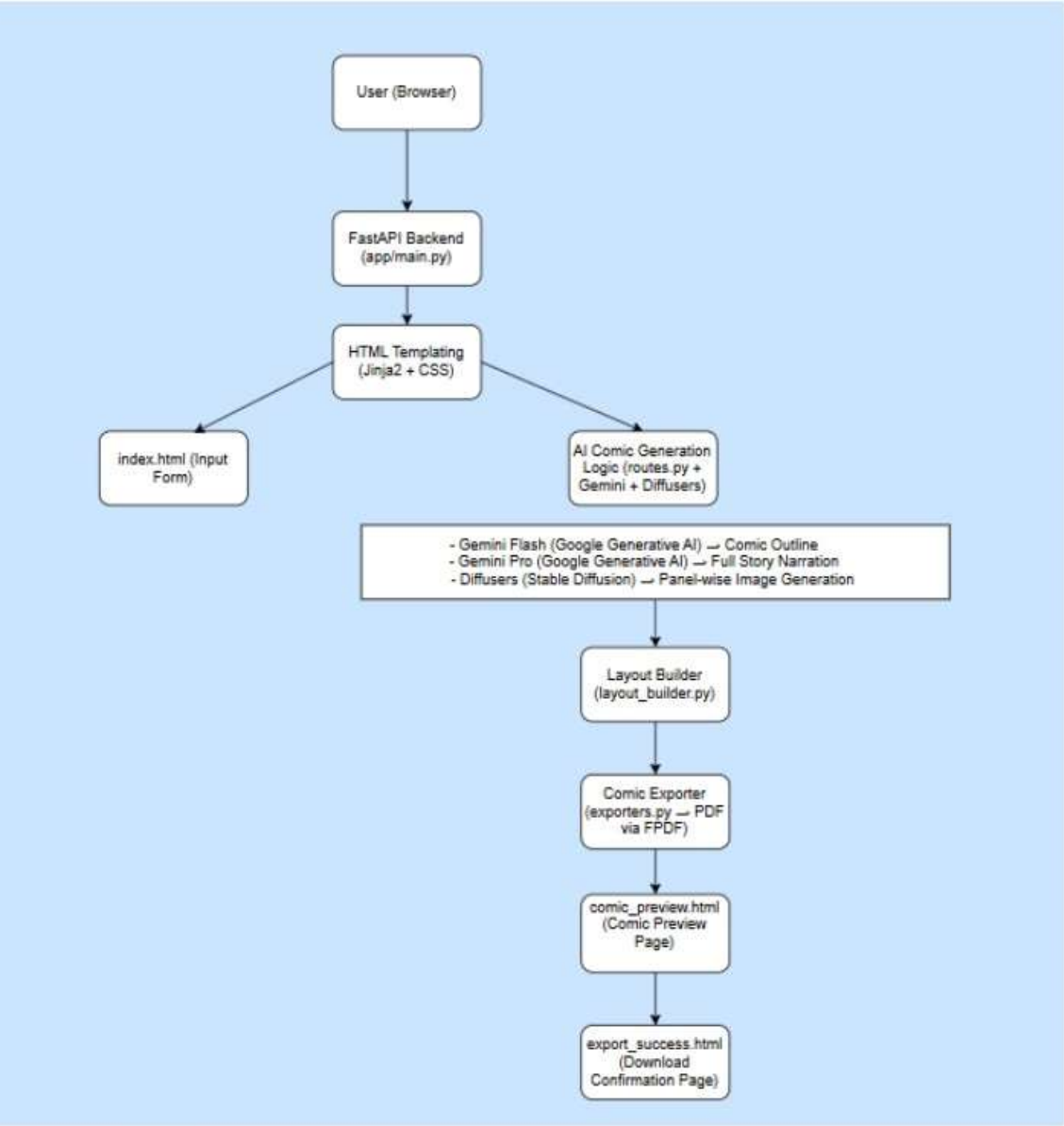
users iterate on and customize the mood and visual style of their comic until it matches their creative vision.

*Scenario 3:* After reviewing the on-screen preview of their comic, the user decides to download it. The system then:

Layout Binding: Uses layout_builder.py to assemble each panel's title, image, and text into a structured layout.

PDF Generation: Calls http://exporters.py to compile the comic into a PDF file using FPDF, placing images and narrative text onto consecutive pages and saving the PDF with a timestamped filename.

Export Success: The user is redirected to an export success page, confirming the successful download. This feature ensures users can save, print, or share their comic in a professional PDF format.




Pre - requisites:

1. FastAPI Framework Knowledge: FastAPI Documentation
2. Uvicorn ASGI Server: Uvicorn Docs
3. Python Programming Proficiency: Python Official Docs
4. HTML, CSS, and Jinja2 Template Skills: W3Schools HTML/CSS/Jinja2 Tutorials
5. Google Gemini API Familiarity: Google Generative AI Documentation

6. Hugging Face Diffusers & Stable Diffusion: Hugging Face Diffusers Documentation

7. PyTorch Basics: PyTorch Documentation

8. PDF Generation with FPDF: FPDF Documentation

9. Version Control with Git: Git Documentation

10. Python Package & Environment Setup (pip, virtualenv, or conda): Virtualenv Guide

## *Project Workflow:*

*Milestone 1. Model selection and Architecture*

Activity 1.1. Research and Select the Appropriate Generative AI Model

Activity 1.2. Define the architecture of the application

Activity 1.3. Set up the development Environment

*Milestone 2. Core functionalities Development*

Activity 2.1. Develop the core functionalities

Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

*Milestone 3. http://route.py Development*

Activity 3.1. Writing the Main Application Logic in http://routes.py

*Milestone 4. Frontend Development*

Activity 4.1. Designing and Developing User Interface

Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2

*Milestone 5. Deployment*

Activity 5.1. Preparing the Application for Local Deployment

Activity 5.2. Testing and Verifying Local Deployment

*Milestone 1. Model selection and Architecture*

In this milestone, we focus on selecting appropriate generative AI models for the ComicCraft application. This involves researching the capabilities and performance of multiple AI models to ensure that the chosen models align well with the application's objectives of generating comic storylines, character dialogues, scene descriptions, and comic-style illustrations.

*Activity 1.1: Research and Select the Appropriate Generative AI Model*

## *Understand the Project Requirements*

Review the specific needs of the ComicCraft application, focusing on the types of content it must generate:

Structured comic outlines (panel-by-panel)

Full comic narration and character dialogues

Visual comic-style illustrations for each panel

The system must deliver creative, engaging, and visually rich comic outputs for users based on their custom story prompts.

## *Model Evaluation*

Research documentation and capabilities of:

Google's Gemini Models: Gemini Flash (optimized for fast structured outputs) and Gemini Pro (optimized for detailed, creative text generation).

Hugging Face Diffusers: Stable Diffusion model (for high-quality AI-generated images based on prompts).

Compare the selected models based on:

Response quality for creative writing tasks (narration and dialogues)

Prompt adherence and richness of detail for scene descriptions

Speed and resolution of image generation

Model stability, available documentation, and community support

Practical testing was conducted using different prompts to assess each model's storytelling abilities and image generation performance.

## *Final Model Selection*

Choose the AI models that best match ComicCraft's project needs:

*Gemini Flash (models/gemini-1.5-flash):* Selected for fast and structured generation of comic panel outlines based on story prompts.

*Gemini Pro (models/gemini-1.5-pro):* Selected for detailed, creative comic narration and engaging character dialogues.

*Stable Diffusion (runwayml/stable-diffusion-v1-5):* Selected for generating high-quality comic-style illustrations from image prompts.

This combination ensures that ComicCraft can deliver rich, creative storytelling alongside vivid, visually appealing comic panels in an automated, efficient manner.

*Activity 1.2. Define the architecture of the application*

*Architecture Overview:* The architecture of ComicCraft is structured into three primary components: Frontend (HTML, CSS, Jinja2), Backend (FastAPI application), AI Services.

The frontend captures user inputs and displays generated comics, the backend handles data processing and routing, and the AI services generate story content and illustrations.

## *Frontend Responsibilities:*

Users interact with a simple and user-friendly web interface developed using HTML, CSS, and Jinja2 templates.

Key user input fields include:

Story Prompt: Main idea for the comic

Character Name: Hero of the comic

Setting: Location (e.g., forest, school, city)

Story Tone: Mood of the story (e.g., dramatic, funny)

Art Style: Visual style preference (e.g., anime, realistic)

## Templates Used:

http://index.html: For user input collection

comic_preview.html: To display the generated comic

export_success.html: To show a success message after comic export

## *Backend Responsibilities:*

FastAPI is used to manage server-side operations and route handling.

Key backend tasks:

Receive form data from the frontend or JSON API calls.

Call AI models to generate panel outlines, story narration, and illustrations.

Organize and build the complete comic layout.

Export the comic to a downloadable PDF format.

Routes are defined in http://routes.py to handle: Form-based and JSON-based comic generation, Image testing, Comic export confirmation.

## *AI Integration Points:*

Gemini Flash is called to generate the structured 5-panel comic outline based on the user's story prompt.

Gemini Pro is called next to create detailed narration and character dialogue from the outline.

Stable Diffusion is used to generate comic-style illustrations based on the AI-generated image prompts for each panel.

*Activity 1.3 Set up the development Environment:*

Install Python & Pip: Ensure that Python is installed on your system along with pip.

Create Virtual Environment:

python -m venv comiccraft-env

comiccraft-env\Scripts\activate  # For Windows

Install Required Libraries:

pip install fastapi uvicorn jinja2 python-multipart google-generativeai diffusers transformers fpdf Pillow accelerate

Run Server:

uvicorn app.main:app --reload

Visit http://127.0.0.1:8000 for the app and /docs for API testing.

*Milestone 2. Core functionalities Development*

*Activity 2.1 Develop the core functionalities:*

### *1. Generate Structured Comic Panel Outline*

Function: generate_outline() File: gemini_flash.py

This function uses the Gemini Flash AI model to generate a structured 5-panel comic outline based on the user's story prompt. Each panel contains a panel number, title, scene description, and an image generation prompt.

### *2. Generate Comic Story Narration and Dialogue*

Function: generate_story() File: gemini_pro.py

This function uses the Gemini Pro AI model to expand the panel outlines into a full comic-style story. It creates engaging narration and character dialogues for each panel.

### *3. Generate Comic Illustrations from Prompts*

Function: generate_image() File: image_generator.py

This function uses the Stable Diffusion model to create a comic-style image based on the provided image prompt. It sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory.

### *4. Organize Comic Panels into Layout Structure*

Functions: build_comic_layout() File: layout_builder.py

This function organizes the generated images and full comic story into a structured layout. It matches each image with its corresponding panel story.

### *5. Export Comic Panel and Story into PDF*

Function: save_pdf() File: http://exporters.py

This function compiles the full comic into a multi-page PDF file using the FPDF library. Each panel's image and narration are placed neatly on separate pages.

*Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing*

*Define Routes in FastAPI:* All routing logic is handled in http://routes.py.

### *Process User Input:*

HTML Form: Created in http://index.html - Users enter: story prompt, main character name, setting, tone, and art style

FastAPI captures inputs using Form(...) parameters from the request

Pydantic schema (PromptRequest) is used to validate JSON input for API-based comic generation

## *Integrate Gemini API Calls:*

/generate → Calls generate_outline() to create panel outlines, Calls generate_story() to expand into narration, Calls generate_image() for each panel, Calls build_comic_layout(), Calls save_pdf()

/generate-comic/json → Similar flow as /generate, but triggered via a raw JSON API request. Returns JSON response with layout data and PDF path.

/test-image → Calls generate_image() directly with a custom prompt to test image generation functionality separately.

*Milestone 3. http://routes.py Development*

In this milestone, the primary focus was on setting up all major FastAPI routes for the app, handling user inputs efficiently, integrating AI generation workflows, and ensuring smooth interaction between backend processing and frontend templates.

*Activity 3.1: Writing the Main Application Logic in http://routes.py*

*Defining Core routes in http://routes.py:*

/ → Loads the homepage (index.html) where users can submit their story details.

/generate → Handles form submission, processes the input using AI, generates the comic panels, and returns the comic preview page.

/generate-comic/json → An API route that accepts JSON payloads, triggers comic generation, and returns comic layout data and the generated PDF path.

/export-success → Displays a success confirmation page after the comic is downloaded.

/test-image → A developer utility route to test image generation from a direct prompt without full comic creation.

*Set up Route Handlers for Each Feature*

For /generate, user inputs are captured using Form(...) parameters for: Story prompt, Character name, Setting, Story tone, Art style

For /generate-comic/json, inputs are captured from a JSON body using a PromptRequest model with Pydantic.

The route handlers manage error handling via FastAPI's HTTPException system and use try-except blocks to gracefully handle unexpected failures like API errors or JSON decode errors.

*Integrate AI workflows in each Route:*

In /generate and /generate-comic/json: generate_outline() is called to create a 5-panel structured story. generate_story() is called to write narration and dialogues. generate_image() is called iteratively to generate images for each panel.

build_comic_layout() structures the final panel layout. save_pdf() compiles the story and illustrations into an exportable PDF.

## *Milestone 4. Frontend Development*

This part involved building responsive and clean web pages using HTML, CSS, and Jinja2 templating.

*Activity 4.1 Designing and Developing the User Interface*

*Set Up the Base HTML Structure:* Developed http://index.html as the main entry point for user input. Structured the form to capture: Story Prompt, Main Character Name, Setting, Story Tone, Art Style.

*Design a Responsive Layout Using CSS:* Embedded CSS styles were applied directly within each HTML file. Layouts use fixed width with centered content to ensure readability. Consistent button styling with hover effects. Input fields and result panels are designed with padding, box shadows, and rounded corners.

## *Create Separate Pages for Each Core Functionality:*

http://index.html: A user-friendly form where users input the story prompt. Submits data to the /generate FastAPI route via POST.

comic_preview.html: Displays the AI-generated comic panel-by-panel. Each panel includes: Title, Scene illustration (image), Scene description (in italics), Captions and narration text.

export_success.html: Displays a success message after the user downloads their comic as a PDF.

*Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2*

*Integrate Jinja2 Templating for Dynamic Content Rendering*

Used FastAPI's Jinja2Templates for template rendering:

from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="templates")

Each FastAPI route uses TemplateResponse to dynamically render HTML templates based on the user's interaction: / route renders http://index.html, /generate route renders comic_preview.html, /export-success route renders export_success.html.

### *Bind Backend Data to HTML Templates*

http://index.html: Form elements use name="..." attributes that directly map to backend Form(...) parameters. Submits form data via POST to /generate.

comic_preview.html: Dynamically displays Panel number, title, Image generated for the panel, Scene description (in italics), Captions and narration text. Loops through the layout list using Jinja2 {% for panel in layout %} syntax.

export_success.html: Displays a confirmation message using Jinja2 variables. Receives the exported PDF path as context.

## *Milestone 5. Deployment*

Now, the focus is on deploying the ComicCraft application on a local system using FastAPI. This milestone verifies that the complete comic generation workflow — including AI text generation, image creation, layout building, and PDF export — runs smoothly.

*Activity 5.1: Preparing the Application for Local Deployment*

Set up a virtual environment:

python -m venv env

# For Windows

env\Scripts\activate

# For macOS/Linux

source env/bin/activate

pip install -r requirements.txt

Configure environment variables:

GEMINI_API_KEY=your_gemini_api_key_here

HF_API_KEY=your-huggingface-api-key-here

*Activity 5.2: Testing and Verifying Local Deployment*

### Start the FASTAPI Server:

uvicorn app.main:app --reload

Once the server is running, open browser and navigate to:

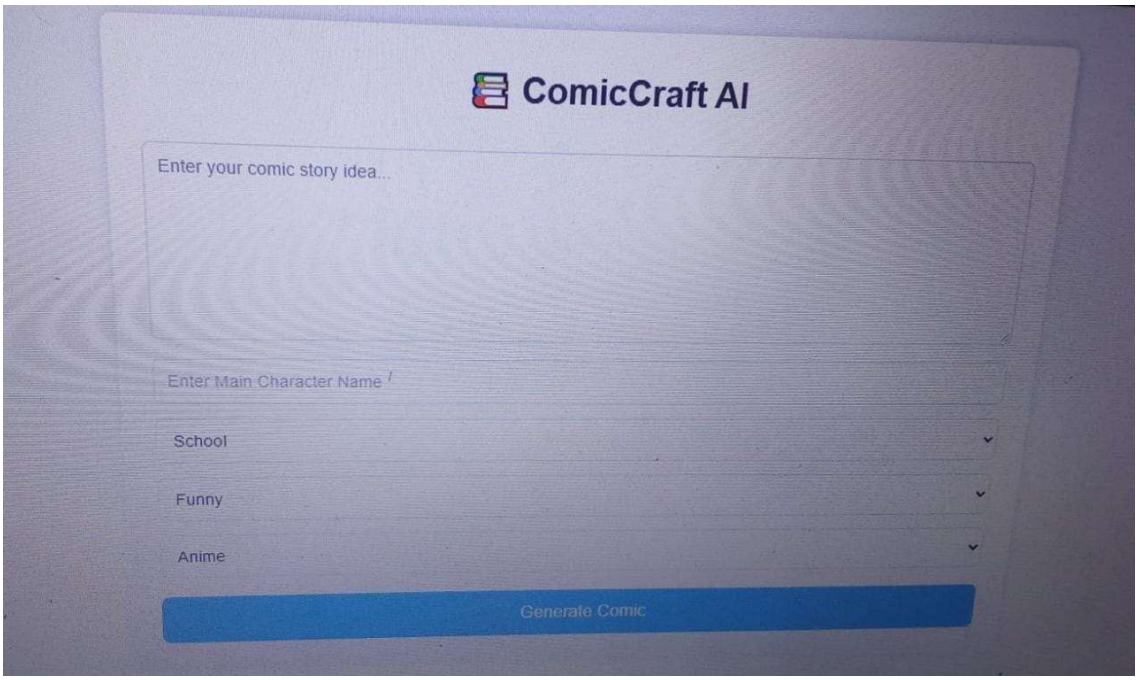
**http://127.0.0.1:8000**

API documentation can be accessed at:

**http://127.0.0.1:8000/docs**

### *Exploring Website's web pages:*

### Home page –




**Description:***

The homepage of ComicCraft is designed as an intuitive starting point for users to create AI-generated comic stories. It features a clean layout with a scenic background that fosters a creative atmosphere. At the center, a structured form collects user inputs necessary for comic generation. Users can: Enter a Story Prompt, Specify a Main Character Name, Choose a Setting (school, forest, space, city), Select a Story Tone (light-hearted, dramatic, poetic, funny), Pick an Art Style (anime, pixel art, comic book, realistic). Once the form is submitted, the system uses AI models to automatically generate a complete comic, including storyline outlines, detailed dialogues, illustrations, and an exportable PDF.

***Comic Preview- Result page –**

## Your Comic Preview

**Panel 1: The Forest's Edge**



*Finn, a red fox with intelligent eyes, stands at the edge of a dark, imposing forest. Sunlight barely penetrates the dense canopy. He looks apprehensive but determined.*

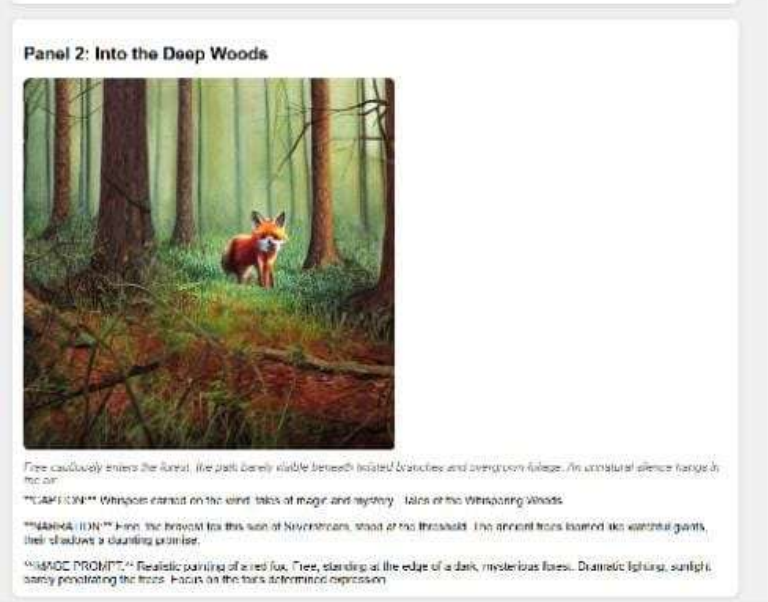

## Your Comic Preview

**Panel 1: The Forest's Edge**





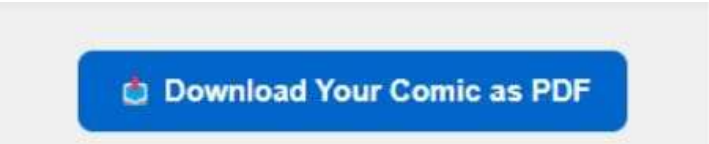
*Finn, a red fox with intelligent eyes, stands at the edge of a dark, imposing forest. Sunlight barely penetrates the dense canopy. He looks apprehensive but determined.*




**Description:***

The Comic Preview Page in ComicCraft presents the AI-generated comic panels sequentially after the user submits their input through the homepage form. Each panel displayed includes: Panel Title, Comic Image: An AI-generated illustration, Scene Description: A short descriptive paragraph in italics, Caption and Narration: Caption providing ambient description and Narration detailing main character's actions, Image Prompt Reference: The original artistic description used to generate the illustration. The design ensures each panel is clearly separated and visually appealing.

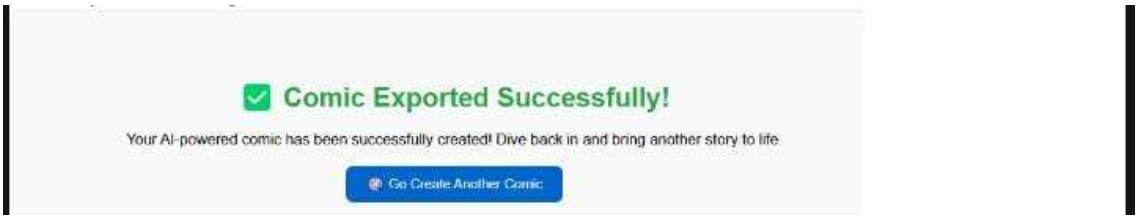
***Download Button –**



### Description:

* On the Comic Preview Page, a "Download Your Comic as PDF" button is displayed. When users click this button, the entire comic — including images, panel titles, scene descriptions, captions, and narration — is compiled into a professionally formatted PDF file and downloaded directly to the user's device.

## Comic Export Success Page:




## *Conclusion:*

ComicCraft successfully demonstrates the creative potential of AI-driven comic generation. By combining the capabilities of Large Language Models (Gemini Flash and Gemini Pro) for storytelling and Stable Diffusion for comic-style illustration, the application enables users to transform simple prompts into complete, visually engaging comic narratives.

The project provides an end-to-end experience — from personalized story outline generation, full narration, high-quality image creation, to exporting the final comic as a downloadable PDF — all within an intuitive web interface.